Document 09 — LangGraph

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

08_rag + Async Python gate

How to Read & Practice This Document

- **What:** building an agent as a clear, visible graph instead of a hidden loop.
- **Why:** real systems need state you can look at, test, and pause — not a black-box `while` loop you can't inspect or replay.
- **When:** any agent complex enough to need branching, saved state, or a human checking in — Project 2's simple loop genuinely didn't need this; Project 3 will.
- **How to practice:** 1. Read the material once, then go read real graphs in the LangGraph repo's `examples/` folder — this tool is best learned from real graphs, not just prose. 2. Do the **Basic/Intermediate** graph exercises closed-book where you can. 3. Rebuild Project 2 as a graph (**Real-world**) — same behavior, new build. Don't add new features until the rebuild is proven correct. 4. Do the **Failure** exercise on purpose — an intentionally-broken loop and a real pause/resume test teach more than reading about them. 5. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say **"Show me the solution."** 6. Before moving on, draw the graph on paper from memory and check it against your code. If they don't match, you're not done.

Jump to: Learn It in Order (start here) · Core Concepts · Practice Exercises · Build Task

The Story — what this document is actually building

Picture this: Project 2's agent (Doc07) works, but it's a black box — a `while` loop hidden inside library code, deciding what to do next. You can't easily look at one single step, test it by itself, or pause it partway through and pick up again later. That's fine for a demo. It's not fine for anything you'd actually want to run, debug, and trust in production.

LangGraph's whole idea is to make that hidden loop *visible* as a graph you build yourself, out of code you own. **State** is one typed shape holding everything the graph is tracking — the task, the conversation, whatever's been found so far. **Nodes** are just functions: state goes in, a small update

comes out. **Edges** say what runs next — a plain edge always goes to the same place, a **conditional edge** picks based on something in state, which is how "should I search or not" branching gets written as real, visible structure instead of a buried `if`. Graphs are even allowed to **loop** back on themselves on purpose — that's how "keep improving until it's good enough" gets built, as long as there's a real stopping rule.

Two more pieces make this genuinely production-ready instead of just a nicer diagram. A **checkpointer** saves the graph's state at every step, so it survives outside any one running program — Doc04's chatbot memory disappeared the moment you closed the terminal; a checkpointed graph's state doesn't. And `interrupt()` uses that saved state to pause the graph mid-run and wait for a human — approve this refund, confirm this email gets sent — before continuing exactly where it left off.

The **Build Task** ties all of this together: you rebuild Project 2's exact tool-using behavior as a graph, add a conditional edge, wire up a checkpointer, and add one real `interrupt()` point. This isn't its own app — it's the literal skeleton Project 3 copies straight into its own `state.py` / `nodes.py` / `graph.py`, then grows into a 3-agent system.

Learn It in Order — Build a Graph Step by Step

New to LangGraph? Start here. Doc07's agent loop was hidden inside a `while`. A graph makes each step an explicit, named, inspectable node — so you can route, loop, and *pause* on purpose. We build one graph one piece at a time. Then the Core Concepts below go deeper, and the Practice Exercises are these exact steps.

> Setup: `pip install langgraph langchain-openai`. Builds on Doc07's loop.

Step 0 — Why a graph at all

Doc07's loop worked, but the flow lived inside a `while` — you couldn't see it, pause it, or draw it. A graph turns each step into a **node** with **edges** between them, so the flow becomes something you can read, test, and stop midway for a human.

Step 1 — State: the typed object that flows through

State is one typed shape listing everything that moves between nodes — like a patient file passed from desk to desk:

```
from typing import TypedDict

class State(TypedDict):
    question: str
```

```
    answer: str
    steps: int
```

What's new: reading the state shape tells you exactly what any node can see or change — no hunting through scattered variables.

Step 2 — A node: state in, a small update out

A **node** is just a function. It returns a small dict of *only the fields it changes* — it never edits state in place:

```
def answer_node(state):
    reply = f"Echo: {state['question']}"
    return {"answer": reply, "steps": state["steps"] + 1}
```

What's new: a node returns an *update*, and the engine applies it. Mutating `state` directly is a bug — always return a dict.

Step 3 — Edges + compile + run

Wire the nodes with edges, compile, and run with `.invoke(state)`:

```
from langgraph.graph import StateGraph, START, END

builder = StateGraph(State)
builder.add_node("answer", answer_node)
builder.add_edge(START, "answer")       # required — nothing runs without this
builder.add_edge("answer", END)
graph = builder.compile()

final = graph.invoke({"question": "hi", "answer": "", "steps": 0})
print(final["answer"], "| steps:", final["steps"])   # Echo: hi | steps: 1
```

What's new: `START` / `END` are imported objects (not strings). The work is the same as a function call — but now it's a named node you can test, draw, and pause.

Step 4 — Conditional edges: a real decision, as structure

A **conditional edge** runs a small function on the state and picks the next node — an `if` that lives in the graph instead of buried three functions deep:

```
def route(state):
    if "refund" in state["question"].lower():
        return "billing"
```

```
    return "general"

# after the "classify" node, go to billing or general:
builder.add_conditional_edges("classify", route,
                              {"billing": "billing", "general": "general"})
```

What's new: `route(state)` returns a **key of the mapping**, and the engine goes to that node. A returned value that isn't in the mapping is a real bug (the Edge-case exercise).

Step 5 — Loops: a conditional edge pointing backward

Point a conditional edge **back** to an earlier node and you have a loop — the same mechanism as Step 4, aimed backward. It **must** have a stopping rule in the state:

```
def keep_going(state):
    if state["steps"] >= 3:          # the stopping rule, in state
        return "done"
    return "again"

builder.add_conditional_edges("work", keep_going,
                              {"again": "work", "done": END})
```

What's new: a loop on purpose, not by accident — the counter lives in state so the stop is visible and checkpointed. (LangGraph's `recursion_limit` is the backstop if you forget.)

Step 6 — Checkpointer + `interrupt()`: pause for a human

A **checkpointer** saves the state at every step; `interrupt()` uses it to pause mid-run and wait for a person, then resume exactly where it stopped:

```
from langgraph.checkpoint.memory import MemorySaver   # prod: SqliteSaver (disk)
from langgraph.types import Command

graph = builder.compile(checkpointer=MemorySaver())
config = {"configurable": {"thread_id": "user-1"}}

graph.invoke(inputs, config)                # runs, then PAUSES at interrupt()
# ... a human reviews the paused state ...
graph.invoke(Command(resume="yes"), config)  # continues from the pause
```

What's new: state now lives *outside* the running program — close the terminal, reopen, and a `SqliteSaver` graph is still paused where it was. This is Project 3's human-approval gate.

You've built a graph, zero to advanced

State → nodes → edges → conditional edges → loops → checkpointer + `interrupt()`. Everything else in this document (parallel branches, subgraphs, streaming, visual debugging, long-term memory) builds on these. Now the Practice Exercises, in order:

You read	Now practice
Steps 1-3 (state, a node, edges)	Basic — your first graph
Step 4 (conditional routing)	Intermediate — real branching
Rebuild Doc07's loop as a graph	Real-world — rebuild Project 2 as a graph
Step 4's trap (an unmapped route value)	Edge cases — an unhandled routing value
Steps 5-6 (loop limit + pause/resume)	Failure — an endless loop, and a real pause/resume

The detailed Core Concepts below explain each piece more deeply — read them after you've run the steps above.

Core Concepts (read this first — everything you need is here)

Topics on this page: LangChain vs. LangGraph vs. RAG · Tool-calling inside a graph · From a hidden loop to a clear graph · State · Nodes and edges · Loops · Checkpointers · `interrupt()` · Time travel & editing state · Parallel branches · Subgraphs · Streaming a graph's execution · Debugging a graph visually · Long-term memory

LangChain vs. LangGraph vs. RAG — how these three actually relate

Doc05 gave you LangChain: building blocks for calling a model — prompt templates, `|`-chained pipelines, output parsers. Doc08 gave you RAG: a technique, not a library, for grounding an answer in your own real documents. This document adds a third piece, and it's easy to think all three compete for the same job, because all three involve "calling a model." They don't. Picture a restaurant kitchen: LangChain is the box of tools on the counter — knife, pan, mixer, each doing one job well. LangGraph is the recipe card taped to the wall — step 1, then step 2, and *if* the sauce is too thin, go back to step 2 again, and *stop and ask the head chef* before serving the expensive dish. RAG is neither a tool nor a recipe — it's the trip to the store room to fetch real ingredients before cooking, so the dish is made from what you actually have. A real app almost always uses all three, at different layers, in the same file.

How it really works

- The order of events in a typical Project-3-style run: your code calls `graph.invoke(...)`; LangGraph reads state and follows the edges you declared to decide which node runs; that node's body is ordinary Python — this is where a LangChain prompt template calls the model, or where Doc08's `retrieve()` runs; the node returns a small dict; LangGraph merges it into state and repeats until `END`.

- Keep model-specific code inside nodes, control flow inside edges. The moment you write `if next_agent == "critic":` inside a node body, that routing decision has escaped into the wrong layer — it's no longer visible in the graph diagram, and can't be tested or replayed on its own.
- **Branching on a decision:** LCEL's `RunnableBranch` picks between a few fixed paths, once. It can't route back to an earlier step or carry its own state — chaining several together to fake a graph gets unreadable fast, because `|` assumes data flows one direction. The hack: a plain `if/elif` around `chain_a.invoke(...)` vs `chain_b.invoke(...)` — a hand-built conditional edge, in code nothing understands as a graph.
- **Looping until done:** LCEL has no "run this chain again based on its own output" primitive — chains run once, start to finish. A `while` loop around `.invoke()` works, but the loop logic is now bespoke Python nothing can inspect, checkpoint, or visualize.
- **Pausing for a human, resuming exactly where it left off:** LangChain has no equivalent — a chain you `.invoke()` runs to completion or raises. You can save state to your own database before the human step and reload it in a second script, but you're manually rebuilding what a checkpointer already does, and it's easy to forget one field that turns out to matter.
- **Surviving a process restart:** same root cause — LCEL keeps its state in Python variables for the life of one `.invoke()` call. The hand-built fix is the same: serialize your own state after every step, and write your own resume logic.
- **Running two things at once and merging results:** LCEL's `RunnableParallel` is a real, working answer here — it runs several chains concurrently and merges them into one dict. It stops being enough the moment the set of parallel branches, or whether to run them at all, is a runtime decision rather than fixed in advance.
- **Remembering a fact across separate, later conversations:** LangChain has no built-in long-term store; its old per-conversation memory classes are now removed, and current code just manages one conversation's message list directly. The hack: your own key-value store (a JSON file, a SQLite table) keyed by user id, read and written by hand — functionally close to `langgraph.store`, minus the namespacing.

Capability	LangChain's answer	LangGraph's answer
Branch on a decision	`RunnableBranch` — one fixed decision, no way back	A conditional edge — any node can route to any other, including back
Loop until done	No native primitive — a manual `while` around `.invoke()`	A real edge back to an earlier node — the loop is part of the graph itself
Pause for a human, resume later	No native primitive — save/reload state by hand	`interrupt()` + a checkpointer — the framework saves and restores state
Survive a process restart	No native primitive — serialize state yourself	A checkpointer (`SqliteSaver` , etc.) — persistence built in
Run steps in parallel	`RunnableParallel` — merges a fixed set of chains, once	Fan-out/fan-in — any number of nodes, conditionally, merged via a reducer

Capability	LangChain's answer	LangGraph's answer
Remember across conversations	No native primitive — build your own key-value store	`langgraph.store` — a namespaced, built-in long-term store

The honest summary: none of this is *impossible* in LangChain — every one of these patterns existed in real production code before LangGraph did, built by hand around LangChain's chains. What LangGraph actually adds is turning each hand-built workaround into a first-class, visible, reusable piece of the framework. That's why this document exists as its own thing, not as one more LangChain feature.

Common mistakes:

- *Mistake:* reaching for LangGraph on day one for a one-prompt script, because it sounds more professional. → *Symptom:* 80 lines of `StateGraph` boilerplate around 4 lines of real work, and every small change means touching three places. → *Fix:* start with the plain function. Move to a graph the first time you need a branch, a loop, a pause, or persistence — not before.
- *Mistake:* treating RAG as a competing framework — "should I use RAG or LangGraph?" → *Symptom:* a question with no real answer, because they aren't on the same axis. → *Fix:* RAG is a technique that becomes one node; Doc08's `retrieve()` drops into a graph unchanged.

Where you'll meet it: Doc10 builds a full search agent as a graph. Project 3 is literally this layering — a LangGraph graph with a Doc08 RAG node inside it, and LangChain prompt templates inside its other nodes. Project 6 — LangChainPro is built to sit deliberately on the LangChain side of this exact line, and its own reading guide cites this topic to explain why. Doc11 and Project 4 keep the same layering with several agents inside one graph. Doc16 and Doc17 both expect you to justify this choice out loud — it's a very common interview question.

Quick cheat sheet:

- LangGraph = control flow. LangChain = one step's work. RAG = a technique, not a library.
- A real app uses all three, at different layers — you almost never pick just one.
- No branch, no loop, no pause, no restart requirement → don't reach for a graph yet.
- `retrieve()` from Doc08 becomes one node, unchanged.
- Keep routing decisions in edges; keep model calls and RAG calls inside nodes.

Tool-calling inside a graph, and the path from one agent to many

One clerk running a small bank branch can open accounts, take cash, and print statements — three tools, one person picking the right one each time. When the branch gets busy, the bank hires a cashier, an account officer, and a loan officer, and they use the exact same three machines the single clerk used — nothing about the machines changed. What gets added is a token counter at the door, deciding which desk a customer goes to. Tool-calling is the machine; multi-agent is the token counter. Doc07 built the machine: a single agent calling tools in a loop. This topic is the mechanism

for running that same machine as one node in a graph — the bridge to running several of them, once Doc11 needs more than one.

How it really works

- One "agent step" inside a graph is always the same six moves, whether there's one agent or ten: the node reads `messages` out of state; it calls the model with tool schemas bound (`model.bind_tools([...])`); the model replies with an `AIMessage` — plain text, a non-empty `.tool_calls` list, or both; the node returns `{"messages": [ai_message]}`, and LangGraph appends it because `messages` carries an `add_messages` reducer; a conditional edge checks the last message — tool calls present routes to the tool node, empty routes to `END`; the tool node runs each call, wraps the result in a `ToolMessage` carrying the matching `tool_call_id`, and an edge sends control back to the agent node. That loop — agent, tools, agent, tools, until the model stops asking — is Doc07's ReAct loop, drawn as real edges instead of hidden inside library code.
- `ToolNode([...])` is the prebuilt version of this: it runs every tool call in the last `AIMessage`, in parallel, and returns matching `ToolMessage`s. It reads `state["messages"]` by that exact name — a differently named field needs `ToolNode(tools, messages_key="chat")`.
- `tools_condition` is the ready-made conditional edge for the routing step above — it checks `.tool_calls` so you don't write that check yourself. It always routes to a node literally named `"tools"` unless you pass your own mapping.
- Parallel tool calls are the default now — a modern model can ask for three tools in one `AIMessage`, and `ToolNode` runs them concurrently. Good for speed, and dangerous for anything with a side effect: three "send email" calls really do send three emails. Gate side-effecting tools behind `interrupt()` (below), or make them idempotent with a request id.
- A failing tool should not crash the graph. By default `ToolNode` catches the exception and returns its text as a `ToolMessage`, so the model can react — the same "return a clear error string" rule as Doc06's failure-handling topic. Pass `ToolNode(tools, handle_tool_errors=False)` for a hard stop instead.
- Retries belong on the node, not inside the tool: `add_node("tools", ToolNode(tools), retry_policy=RetryPolicy(max_attempts=3))` reuses Doc02's backoff discipline, but now a flaky call is retried by the framework and shows up in the trace.
- Tool ownership is the safety boundary in a multi-agent graph: each agent node binds only its own tools. A prompt injection reaching a research agent cannot issue a refund, because that agent never bound the refund tool — the model can't talk its way into a tool call it has no schema for.

```
# risky_tool_routing.py — one risky tool gated behind an approval node
RISKY = {"issue_refund"}

def route_tools(state) -> str:
    last = state["messages"][-1]
    if not getattr(last, "tool_calls", None):
```

```
        return "END"
    has_match = False
    for c in last.tool_calls:
        if c["name"] in RISKY:
            has_match = True
    if has_match:
        return "human_approval"
    return "safe_tools"

builder.add_conditional_edges(
    "agent", route_tools,
    {
        "safe_tools": "safe_tools",
        "human_approval": "human_approval",
        "END": END,
    },
)
```

Ordinary lookups flow straight through; the moment the model asks for `issue_refund`, control goes to a node that pauses for a person — a check in your Python, not a polite line in the prompt.

Situation	What to do	Why
Standard agent, standard loop	`create_agent(model, tools)` (Doc07)	Shortest correct code — it already compiles to a LangGraph graph
Standard loop, plus one extra node (approval, logging, RAG)	Hand-build with `ToolNode` + `tools_condition`	You need to own the edges to insert a node between them
Every tool call must hit an audit log	A hand-written tool node	`ToolNode` gives no hook before each call runs
Two agents, different tool sets	Two agent nodes, two `ToolNode`s, one router	Each specialist keeps a small, focused tool list — Doc11's territory
One risky tool (refund, send email)	Route it through a conditional edge to an approval node	The check lives in your Python, not a polite line in the prompt

Common mistakes:

- *Mistake:* giving every agent in a multi-agent graph the full tool list "so they can help each other." → *Symptom:* agents doing each other's jobs, and a refund issued by the research agent. → *Fix:* one tool list per agent node, as small as the job allows — a tool never bound is a tool never callable.

- *Mistake:* forgetting the edge from the tool node back to the agent node. → *Symptom:* the tool runs, the result lands in state, and the graph ends without ever answering. → *Fix:* `builder.add_edge("tools", "agent")` — the loop back is what turns a one-shot call into an agent.

Where you'll meet it: Doc06's round trip and Doc07's ReAct loop are exactly the mechanism running inside one node here — the Real-world exercise below is the direct translation drill. Doc10 adds retries and fallbacks around the same loop. Doc11 turns one agent node into many, with tool ownership as the safety rule. Project 4 and Project 11 — MCPCrew both run this exact pattern with MCP-provided tools instead of local ones.

Quick cheat sheet:

- Tool-calling in a graph is Doc06/Doc07's mechanism — only the wrapper changed.
- `ToolNode` runs the tools; `tools_condition` is the ready-made "tools or END" edge.
- Always add the edge from the tool node back to the agent node.
- One tool list per agent node — a tool never bound is a tool never callable.
- Risky tools get their own node behind an approval gate, not a line in the prompt.

From a hidden loop to a clear graph

Doc07's `create_agent` loop hides its own decisions inside library code — a courier boy who disappears with your parcel and comes back twenty minutes later saying "delivered," with no way to know which street he took or whether he stopped for tea. A `StateGraph` is the same delivery with a printed route sheet instead: numbered stops, a scan at each one. Same delivery, same boy — but now you can watch it, check one stop on its own, and restart from stop 4 if the bike breaks down. That visibility, not style, is the whole point of building a graph instead of a loop.

How it really works

- A `StateGraph` runs your nodes in rounds LangGraph calls a **superstep**: `add_node` / `add_edge` only draw the map — nothing runs yet; `compile()` validates it (every node reachable? every conditional target real?) and returns a runnable; `invoke(input)` starts the engine, which puts the nodes connected to `START` into a "ready" set; every ready node in a superstep runs, together if more than one is ready; each finished node returns a small dict, and the engine merges it into state using each field's reducer; if a checkpointer is attached, the new state is saved right here, between supersteps; the engine follows the outgoing edges to compute the next ready set and repeats until `END`.
- Nodes never call each other. A node returns; the engine decides what runs next. This is why a checkpoint exists between every pair of nodes, and why pausing, resuming, and replaying a run from any step are all possible at all.
- `compile()`'s validation catches real bugs early — a dangling edge or an unreachable node fails at compile time, not at 3 AM in production. Treat a compile error as a free bug catch.

- Build and compile the graph once, at import time. In a FastAPI service, `graph = build_graph()` belongs at module level; each request calls `graph.invoke(...)` with its own `thread_id`. Compiling per request wastes time and, with an in-memory checkpointer, gives every request a brand-new saver — which quietly breaks resume.
- A compiled graph is safe to share across requests; state is not. The graph object holds no run state — everything belonging to one run lives in the state dict and the checkpointer, keyed by `thread_id`.
- Node granularity is a design decision: too coarse (one giant node doing everything) and you're back to a hidden loop with extra ceremony; too fine (a node per line) and the diagram is noise. A workable rule: a node is one thing that could fail on its own, or one thing you'd want to retry or checkpoint alone. A model call is a node; a `.strip()` is not.

```
# service.py — build once, run many
from fastapi import FastAPI
from langgraph.checkpoint.memory import InMemorySaver
from my_graph import builder                  # StateGraph builder, defined once

app = FastAPI()
# built ONCE, at import time
GRAPH = builder.compile(checkpointer=InMemorySaver())

@app.post("/run")
def run(task: str, thread_id: str) -> dict:
    config = {"configurable": {"thread_id": thread_id}}
    out = GRAPH.invoke({"task": task}, config)
    return {"answer": out["answer"]}
```

One graph object serves every caller; each caller's run is kept apart by `thread_id`. Swap `InMemorySaver` for a database-backed saver and the same file survives a restart with no other change.

Situation	What to do	Why
One prompt, one answer	Plain function	A graph adds 30 lines for zero value
Tool loop, no pause, no persistence	`create_agent` (Doc07)	Already a graph inside, one line to write
A real branch ("search or answer directly")	Graph + conditional edge	The decision becomes visible and testable on its own
"Improve until good enough"	Graph + loop-back edge + counter	The stopping rule is part of the structure, not buried in an `if`
Must pause for a human, or survive a restart	Graph + checkpointer	Nothing else in the stack saves state between steps

Situation	What to do	Why
Very long-running, mostly non-LLM jobs (hours)	A workflow engine (Temporal, Airflow)	Built for durability at that time scale — not this document's territory

Common mistakes:

- *Mistake:* putting all the work in one giant node "for now." → *Symptom:* the diagram shows one box, checkpoints are useless, and one failure loses the whole run. → *Fix:* split at every point that could fail alone or that you'd want to retry alone.
- *Mistake:* compiling the graph inside the request handler. → *Symptom:* slow requests, and with an in-memory checkpointer, resume silently never works because each request got a new saver. → *Fix:* compile once at module level; pass `thread_id` per call.

Where you'll meet it: the Basic exercise below is your first `StateGraph`; the Real-world exercise is this exact translation, loop to graph. Doc10 grows this skeleton into Project 3. Doc13 tests node functions one at a time, only possible because they're separate. Doc14 debugs graphs whose edges are wrong. Every multi-agent project (4, 10, 11) is built on this.

Quick cheat sheet:

- Nodes do work; edges decide order; nodes never call each other.
- One superstep = run ready nodes → merge updates → save checkpoint.
- `compile()` validates the map — treat its errors as free bug-catching.
- Build and compile the graph once, at import time; pass `thread_id` per run.
- A node is one thing that can fail, retry, or checkpoint alone.

State: one typed object flowing through the graph

A patient file moves from reception to doctor to lab to pharmacy. Nobody carries the patient's history in their head — everything is written in the file, and each desk fills in only its own section before passing it on. A graph's **state** is that file: one typed shape (a `TypedDict` or Pydantic model) listing every piece of data that can move between nodes. Reading the state shape tells you exactly what any node could possibly see or change — no tracing scattered variables through hidden loop code.

How it really works

- A node does not modify state — it returns a small dict describing a change, and the engine applies it. Anything mutated in place inside a node is a bug waiting to happen; treat incoming state as read-only.
- For each key a node returns, the engine looks up that field's **reducer**: no reducer means the new value replaces the old one; a reducer function (`operator.add`, `add_messages`, or your own) means the engine calls `reducer(old, new)` and stores the result. This is the single most important idea in the topic — it answers "what happens when two things write the same field," unavoidable the moment you add parallel branches or a message list.

- `Annotated[list, operator.add]` appends instead of replacing — the fix when several nodes contribute findings. `Annotated[list, add_messages]` appends chat messages and replaces one with a matching id — use it only for message objects, never plain strings.
- A custom reducer must be pure and defensive: it may be called with `None` as the old value on the very first write, and it must not mutate its inputs, because an older checkpoint may still reference them.
- State is written to storage on every superstep — size is a direct cost, in bytes and latency. A graph carrying 20 retrieved documents' full text through 10 steps writes that text 10 times. Keep bulky data outside state and reference it by id or path; this is the single most common performance problem in real LangGraph apps.
- Secrets never belong in state. Checkpoints are rows in a database that backups, operations staff, and support tooling can all read — the same "never log a secret" discipline as Doc01's logging topic, one layer up. Use runtime context or a module-level client for anything sensitive.
- Schema changes break old checkpoints: renaming `notes` to `findings` leaves every paused run on disk with the old key. Additive changes are safe; renames need a migration, or a release that reads both keys for a while.
- In a multi-agent system, state is the contract between agents — Doc11's supervisor reads `next_agent`; the writer reads `plan` and writes `draft`; the critic reads `draft` and writes `critique`. Writing that shape down before any agent code is the single most effective way to stop a multi-agent project turning into mud.

Situation	What to do	Why
The next node needs it	Put it in state	That's exactly what state is for
Only this node needs it, briefly	A local variable	State is the graph's public surface — keep it small
The whole conversation	One `messages` field with `add_messages`	Every LangChain/LangGraph helper expects this exact name and shape
An API key or DB connection	Not in state — runtime context or a module-level client	State is checkpointed to disk; secrets must not be
A large file (a PDF, an image)	Store the file, put the path/id in state	Every checkpoint would otherwise copy the whole blob
A loop-limit counter	In state, with `operator.add` or `+ 1`	The stopping rule must be visible and checkpointed

Common mistakes:

- *Mistake:* mutating state in place (`state["notes"].append("x")`) instead of returning an update. → *Symptom:* it appears to work locally, then values double-apply, or a restored

checkpoint doesn't match the trace. → *Fix:* always `return {"notes": ["x"]}` and let the reducer do the joining.

- *Mistake:* a plain `list` field written by several nodes, with no reducer. → *Symptom:* earlier findings vanish (the last writer replaced the whole list), or an `InvalidUpdateError` on parallel writers. → *Fix:* add a reducer — `Annotated[list, operator.add]`, or a custom one for de-duplication.

Where you'll meet it: the Basic exercise defines your first `TypedDict` state, and the Build Task puts it in its own `state.py` on purpose. Doc10 grows that state as Project 3 grows. Doc11 treats state as the contract between agents. Doc13 tests nodes by passing in a hand-made state dict — easy because state is one plain, typed shape.

Quick cheat sheet:

- Nodes return a small update; they never edit state in place.
- No reducer = replace. Reducer = merge. Choose deliberately, per field.
- `add_messages` for conversations, `operator.add` for plain lists and counters.
- Keep state small — it's written to storage on every step.
- Ids and paths in state; blobs on disk; secrets in runtime context, never state.

Nodes and edges, including conditional edges

A **node** is just a function: state goes in, a small update comes out. It does one piece of work — call a tool, call the model, check something — and nothing more. An **edge** connects nodes and says what runs next. A plain edge always goes to the same next node; a **conditional edge** instead runs a small function against the current state and picks the next node based on the answer. This is how a real decision — search or don't, approve or reject — becomes visible graph structure instead of an `if` buried three functions deep.

How it really works

- `add_node(name, fn)` registers one unit of work under a unique string name — a typo in a later reference to that name only surfaces as a "node not found" error, and only at `compile()`, not when you write the line.
- `add_edge(a, b)` is a fixed, unconditional hop. `add_edge(START, "first_node")` is required — nothing runs without an edge out of `START`.
- `add_conditional_edges(node, route_fn, mapping)` calls `route_fn(state)` after `node` finishes, and sends control to whichever key of `mapping` the function returned. Omit `mapping`, and `route_fn` must return real node names directly instead of mapping keys.
- `route_fn` returning a value that isn't a key in `mapping` (and isn't a valid node name when `mapping` is omitted) is a real, common bug — see the Edge cases exercise for exactly what LangGraph does about it.

- A node's function signature is the whole contract: it takes the current state (a plain dict at run time, whatever your `TypedDict` says at edit time) and returns a dict of only the keys it wants to change. Nothing about a node's return value tells LangGraph what to do next — that decision lives entirely in the edges leaving it.
- `START` and `END` are objects imported from `langgraph.graph`, not the strings `"start"` / `"end"` — a common beginner typo that fails at `compile()` with a clear error, which is the cheapest place for it to fail.
- A conditional edge can point back to a node earlier in the graph — this is exactly how a real loop (the next topic) gets built: the same mechanism, aimed backward instead of forward.

Piece	What it does	Trap
`add_node(name, fn)`	Registers one unit of work	Names must be unique strings; a typo surfaces only at `compile()`
`add_edge(a, b)`	Always go from a to b	`add_edge(START, ...)` is required, or nothing runs
`add_conditional_edges(a, fn, map)`	Ask `fn(state)` where to go next	`fn` returns a key of `map`, not a node name, unless `map` is omitted
`START` / `END`	The two special node names	Imported objects, not the strings `"start"` / `"end"`
A conditional edge pointing backward	A real loop	Needs a stopping rule — see the next topic

Common mistakes:

- *Mistake:* reading `state["messages"][-1].tool_calls` without checking it exists. → *Symptom:* `AttributeError` on a plain text reply, which is the normal ending case. → *Fix:* `getattr(last, "tool_calls", None)`, or use the prebuilt `tools_condition`.
- *Mistake:* a routing function returning a string that isn't in the edge's mapping. → *Symptom:* a confusing runtime error far from the actual typo. → *Fix:* keep the mapping's keys and the function's return values side by side while writing both, and cover the unmapped case in a test — see the Edge cases exercise.

Where you'll meet it: this is the mechanism every "should I search / should I call this agent" decision in the rest of the curriculum is built from — Project 3's "search or don't" routing, and Project 4's supervisor routing. Doc11's supervisor pattern is nothing more than "agents are nodes, the supervisor is a conditional edge."

Quick cheat sheet:

- A node: state in, small update out, one job.
- A plain edge: always the same next node. A conditional edge: `route_fn(state)` decides.

- `add_edge(START, ...)` is mandatory; `START` / `END` are imported objects, not strings.
- A conditional edge pointing backward is how a loop gets built — the next topic covers the stopping rule it needs.

Loops: on purpose, not by accident

Unlike a one-way flowchart, a LangGraph graph is allowed to loop back on itself — a conditional edge can send control to a node earlier in the graph. This is exactly how "keep improving until it's good enough" gets built, and it needs a real stopping rule for the same reason Doc07's agent loop did: a graph structure does not protect you from an endless loop by itself. It just makes the loop visible in the diagram instead of hidden inside library code — visible is not the same as safe.

How it really works

- A loop is nothing more than a conditional edge whose mapping includes an earlier node's name. Nothing in `compile()` checks that a loop actually terminates — that check does not exist, and can't exist in general, so it's entirely your job.
- The same three-limit discipline from Doc07's step-limit topic applies here: a counter in state (`Annotated[int, operator.add]`), a wall-clock deadline, and — specifically in LangGraph — `recursion_limit`, passed in the run's `config`.
- `recursion_limit` counts graph steps, not "rounds" of your loop. A single agent-plus-tools round trip costs roughly two graph steps (agent node, then tool node), so `recursion_limit=10` is about five rounds, not ten — the same counting trap Doc07 names for `create_agent`.
- When the limit fires, LangGraph raises `GraphRecursionError`. Catching it and returning a generic message with nothing logged turns a fixable bug into an invisible one — log the state at the moment it fired, the same rule Doc07 gives for `MaxIterationsExceeded`.
- A loop's own counter belongs in state, not in a Python variable closed over by the node function — a variable outside state is not checkpointed, so a paused-and-resumed run silently forgets how many times it already looped.

Situation	What to do	Why
"Keep improving until good enough"	A conditional edge back to an earlier node, plus a counter field in state	The stopping rule must be visible and checkpointed, not just "eventually the model stops"
Loop count must survive a pause/resume	Counter in state (`operator.add`), never a closed-over Python variable	Only state is saved by the checkpointer
Defense in depth	State counter **and** `recursion_limit` in config	Each catches a failure the other misses — same layering as Doc02's timeout-plus-retry-cap

Situation	What to do	Why
The limit fires	Raise/log with the partial state attached	A silent generic message hides which node was stuck

Common mistakes:

- *Mistake:* trusting `recursion_limit` alone, with no counter in state. → *Symptom:* `GraphRecursionError` with no readable record of what the graph was doing when it fired. → *Fix:* keep your own counter in state too, so the error can say something useful.
- *Mistake:* assuming `recursion_limit=10` means 10 rounds of your loop. → *Symptom:* `GraphRecursionError` after only about five real rounds. → *Fix:* count roughly two graph steps per round; set `recursion_limit` to about `2 × desired_rounds + 2`.

Where you'll meet it: Doc07's step-limit topic is the same discipline one layer down, before there was a graph to hang it on. Doc11's writer-and-critic pattern is this loop with a model judging "good enough" on each pass. The Failure exercise below makes you watch an unguarded loop hit its limit on purpose.

Quick cheat sheet:

- A loop is a conditional edge pointing backward — nothing checks it terminates for you.
- Layer three limits: a state counter, a wall-clock deadline, and `recursion_limit`.
- `recursion_limit` counts graph steps — roughly two per agent-plus-tools round.
- Put the loop counter in state, never in a Python variable outside it.

Checkpointers: saved state that survives a pause

A checkpointer saves the graph's state at the end of every superstep — in memory while testing, in a real database in production. Without one, a graph that stops has simply lost everything it knew. Doc04's chatbot "memory" only ever existed as a Python list inside one running program — kill the program, and it's gone. A checkpointed graph's state exists on its own, outside any one running process — the same discipline Doc01's config and `.env` topic already asked of your secrets: things that matter shouldn't live only inside a program that might restart.

How it really works

- A checkpoint is saved automatically, with no extra code in your nodes, at the end of every superstep — right after the engine merges each node's returned update into state (see From a hidden loop to a clear graph).
- Every run needs a `thread_id`, passed as `config={"configurable": {"thread_id": "..."}}`. The checkpointer keys everything by this id — one conversation's state never leaks into another's, which is also why it can't double as long-term, cross-conversation memory (see Long-term memory below).

- `InMemorySaver()` (the exercises use its other name, `MemorySaver` — the same class) is correct for tests and local development — and actively wrong for anything that must survive a restart, since it's exactly the Python-list-in-a-running-process problem this topic exists to fix. `SqliteSaver` / `PostgresSaver` are the production choices.
- Building the graph with `compile(checkpointer=...)` once at module level, and passing a **different** `thread_id` per conversation, is what lets one process serve many users at once. Compiling per request gives every request its own fresh in-memory saver, which quietly breaks resume — the single most common cause of "my chatbot works locally but forgets everything in production."
- A checkpoint captures the **whole state dict**, not a diff — exactly why the State topic warns against putting large blobs or secrets in state: every superstep writes the whole thing again.
- Because a checkpoint exists between every pair of nodes, LangGraph can also resume a run from an **earlier** checkpoint instead of the latest one — useful for retrying from before a bad step instead of from the very start.

Saver	Runs where	Use when
`InMemorySaver()`	In the running process, gone on exit	Tests, local development — never production
`SqliteSaver`	A local `.sqlite` file	A single-server app that must survive a restart
`PostgresSaver`	Your existing Postgres	Multiple servers, or you already run Postgres for everything else
No checkpointer	Nowhere — nothing is saved	A graph that never needs pause, resume, or restart survival

Common mistakes:

- *Mistake:* compiling the graph inside a request handler with `InMemorySaver()`. → *Symptom:* works perfectly in a quick local test, then "forgets" everything between two real API calls. → *Fix:* compile once at module level; give each caller their own `thread_id`, not their own graph.
- *Mistake:* assuming an in-memory checkpointer "basically" survives a restart because it worked fine all through testing. → *Symptom:* a production incident the first time the process actually restarts. → *Fix:* use a database-backed saver (`SqliteSaver` / `PostgresSaver`) for anything that must survive one.

Where you'll meet it: Doc01's config/logging discipline is the same "must survive outside one running program" instinct, one layer down; Doc03's "no memory inside the model" is the fact a checkpointer exists to work around. Project 3's Step 2 connects a checkpointer to a rebuilt Project 2 agent and proves resume across a fresh process, not just the same one still running. Project 10 — MemoryKeeper builds the full checkpointer-plus-long-term-store system end to end.

Quick cheat sheet:

- A checkpoint is saved automatically, after every superstep, keyed by `thread_id`.
- `InMemorySaver` for tests only; `SqliteSaver` / `PostgresSaver` for anything real.
- Compile once, with the checkpointer, at module level — never per request.
- One `thread_id` per conversation; a checkpointer never bridges across threads by itself.

`interrupt()`: pausing for a human

Some actions matter enough that a person should sign off before the graph continues — sending a real email, issuing a refund, deleting a record. Calling `interrupt()` inside a node pauses the graph right there, saves the state through the checkpointer, and hands control back to whatever is running the graph. A later, separate call resumes it from exactly that point, using the saved state — no flags, no polling, no hand-rolled "come back tomorrow and check a database column" logic.

How it really works

- `interrupt()` only works with a checkpointer attached — the pause is really "stop, and rely on the last saved checkpoint to know where to resume." No checkpointer, no real pause.
- Calling `interrupt(payload)` raises a special exception internally that LangGraph catches; the payload — whatever you pass, usually a description of the pending action — is what a human-facing UI reads to decide what to show. `graph.invoke(...)` on a run that hits an interrupt returns with the run paused, not with an error your code has to catch.
- Resuming uses the same `thread_id` and `Command(resume=value)`: `graph.invoke(Command(resume=True), config)`. The node that called `interrupt()` re-runs from its own start, with `interrupt()` now returning `value` instead of pausing again — write that node so everything before the `interrupt()` call is safe to repeat.
- This is why the risky-tool-routing pattern from Tool-calling inside a graph sends control to a **separate** node for the approval gate, rather than calling `interrupt()` in the middle of the agent node — the whole agent node re-running on resume could re-ask the model and pick a different tool.
- A rejected approval is not a special LangGraph feature — it's an ordinary state field. `Command(resume={"approved": False})` and a conditional edge reading that field is the whole mechanism; there's no separate "reject" primitive to learn.
- The model can't talk its way past an `interrupt()` gate placed in your Python. A prompt can't skip a pause your code inserted, unlike an instruction in a system prompt asking the model to "please confirm before doing X" — which is a request, not a gate.

Situation	What to do	Why
A tool that spends money or sends something real	Route it to a node behind `interrupt()`	The model's confidence is not consent
A tool that only reads data	No gate needed	Nothing irreversible happens if it runs

Situation	What to do	Why
The human rejects the action	A state field (`approved: bool`) plus a conditional edge	`interrupt()` has no separate "reject" primitive — it's ordinary state
No checkpointer attached	`interrupt()` cannot really pause	The pause depends entirely on a saved checkpoint to resume from

Common mistakes:

- *Mistake:* calling `interrupt()` inside the same node that also calls the model and picks a tool. → *Symptom:* resuming re-runs the whole node, and the model can pick a different tool than the one that was actually approved. → *Fix:* route to a dedicated approval node — decide, then gate, as two separate nodes.
- *Mistake:* testing pause/resume only in the same running process. → *Symptom:* it works in the demo and then loses state the first time it's actually needed, because the process never really restarted. → *Fix:* test resume from a **second, fresh process** with a database-backed checkpointer, exactly as the Failure exercise requires.

Where you'll meet it: Project 3's Approval agent is this exact gate, and its final step is where the full search → answer → pause → resume cycle comes together. Doc11 uses the same gate for any agent with a consequential tool. The Build Task below requires one real `interrupt()` point.

Quick cheat sheet:

- `interrupt()` needs a checkpointer — the pause is really "stop and trust the last saved state."
- Resume with the same `thread_id` and `Command(resume=value)`.
- Put the gate in its own node — don't interrupt inside a node that also decides what to do.
- Rejection is ordinary state (a bool field), not a separate primitive.

Time travel and editing a paused run

Because a checkpointer saves state after **every** step, you don't only have the *latest* state — you have every past one. That gives you two powerful moves on a paused or finished run: **look back** at (and re-run from) any earlier step, and **edit** the current state before resuming. This is what makes a stuck agent debuggable instead of a mystery, and what lets a human *fix* a run rather than only approve or reject it.

How it really works

- `graph.get_state(config)` returns the **current** `StateSnapshot` — `.values` (the state dict), `.next` (which node runs next; empty means finished), and `.config` (which carries a `checkpoint_id`).
- `graph.get_state_history(config)` returns **every** snapshot for that thread, newest first — the full timeline of the run, one entry per step.

- **Time travel:** take an earlier snapshot's `.config` (it names that checkpoint) and pass it back to `graph.invoke(None, that_config)` to re-run **from that point** — optionally after changing something, which forks a new branch of the run instead of overwriting history.
- **Editing state:** `graph.update_state(config, {"field": new_value})` writes an update into the paused state exactly as if a node had returned it (reducers still apply). Resume afterward and the graph continues with your correction.
- Editing is the third human-in-the-loop option: a plain `interrupt()` lets a human **approve or reject**; `update_state` lets them **fix** — change the draft, correct a wrong tool argument, drop a bad retrieved chunk — then let the run finish.
- All of this needs a checkpointer and a `thread_id`; it's reading and rewriting the same saved checkpoints `interrupt()` pauses on.

```
# every past step of this thread, newest first
history = list(graph.get_state_history(config))
print(len(history), "checkpoints")          # one per step

# edit the paused state, then resume
graph.update_state(config, {"draft": "the human-corrected draft"})
graph.invoke(None, config)                  # continues with the correction

# time travel: re-run from an earlier checkpoint
earlier = history[-1].config                # an older snapshot's config
graph.invoke(None, earlier)                 # replays from that point
```

Common mistakes:

- *Mistake:* trying to "go back" by mutating `.values` from `get_state`. → *Symptom:* nothing changes — you edited a copy. → *Fix:* `update_state(config, {...})` is the only write path; it goes through the reducers and saves a new checkpoint.
- *Mistake:* expecting history without a checkpointer. → *Symptom:* `get_state_history` is empty. → *Fix:* history only exists because the checkpointer saved it — no checkpointer, no past.

Where you'll meet it: Doc14's debugging drills use `get_state_history` to see exactly where a run went wrong; a real human-in-the-loop system (Project 3's approval) can grow from "approve/reject" to "edit, then approve" with `update_state`.

Quick cheat sheet:

- `get_state(config)` = now; `get_state_history(config)` = every past step.
- `update_state(config, {...})` edits a paused run (reducers apply); then resume.
- Pass an older snapshot's `.config` to `invoke(None, ...)` to re-run from there.
- Interrupt = approve/reject; `update_state` = fix. Both need a checkpointer.

Parallel branches (fan-out / fan-in)

A graph doesn't have to run one node after another in a single line. One edge can fan out to two or more nodes at once, let each do its own work, and fan back in to one shared node that waits for all of them. Checking a price API and a weather API one after another wastes time waiting for the first before the second even starts; running them in parallel means the graph only waits as long as the slower one takes.

How it really works

- You connect one node to two or more nodes with plain edges, and connect all of those forward to one shared node — LangGraph runs the parallel nodes concurrently and merges their state updates before the shared node runs.
- If two parallel branches write the **same** field, the reducer question from State becomes unavoidable: with no reducer, simultaneous writes either raise `InvalidUpdateError` or silently keep whichever update the engine applied last — neither is what you want. Give the field a reducer (`operator.add`, or a custom merge function) and both branches' updates combine instead of one clobbering the other.
- This is the graph-shaped version of Doc08b's `asyncio.gather` fan-out/fan-in: `gather` runs coroutines concurrently and waits for all of them; a graph's fan-out runs nodes concurrently and waits for all of them before the fan-in node runs. LangGraph does the scheduling, but the underlying idea — concurrent work, collected once everything finishes — is the same one.
- A slow branch is the one the whole fan-in waits for, exactly like `gather` — total time is the slowest branch's time, not the sum of every branch. That's the entire point of fanning out instead of running sequentially.
- One branch failing does not automatically cancel the others by default; decide per graph whether a fan-out is "wait for all, then decide" or "stop the moment one fails" — the same trade-off Doc08b's gather topic covers with `return_exceptions`.
- In a multi-agent system, fan-out is how independent specialists (a researcher and a fact-checker with no dependency on each other) run at the same time instead of waiting in a queue — Doc11 is where this becomes a routing decision, not just a performance trick.

```
builder.add_edge("start", "lookup_price")
builder.add_edge("start", "lookup_weather")
builder.add_edge("lookup_price", "combine_results")
builder.add_edge("lookup_weather", "combine_results")
```

Situation	What to do	Why
Two independent lookups, no shared field	Fan out with plain edges, fan in to one node	Nothing to merge — order of arrival doesn't matter

Situation	What to do	Why
Two branches writing the same field	Add a reducer (`operator.add` , or a custom merge)	No reducer means a clobber or an `InvalidUpdateError`
One branch is much slower than the others	Fan out anyway	The wait is the slowest branch's time either way — sequential would be worse
A failing branch must stop the whole run	Check and route after fan-in, not inside one branch	Fan-in already collects every branch's result to inspect

Common mistakes:

- *Mistake:* two parallel branches writing to the same plain field with no reducer. → *Symptom:* `InvalidUpdateError`, or one branch's result silently disappears. → *Fix:* give the field a reducer before adding the second writer — `operator.add` for simple cases, a custom merge for de-duplication.
- *Mistake:* fanning out branches that all depend on the same slow input, expecting a speed win. → *Symptom:* no measurable improvement over running them sequentially. → *Fix:* fan-out only helps genuinely independent work — the same rule Doc08b gives for `gather`.

Where you'll meet it: Doc08b's `asyncio.gather` topic is the same fan-out/fan-in idea one layer down, without a graph engine doing the scheduling. Doc06's parallel tool calls is the same fan-out inside a single agent turn, before there's a whole graph around it. Doc11 uses fan-out to run independent specialists concurrently instead of in a queue.

Quick cheat sheet:

- Fan out with plain edges to several nodes; fan in by pointing them all to one next node.
- A field two branches both write needs a reducer, or one clobbers the other.
- Total wait = the slowest branch, same rule as `asyncio.gather` one layer down.
- Decide per graph: wait for all branches, or stop the rest on first failure.

Subgraphs: a graph as one node in a bigger graph

A subgraph is a whole compiled `StateGraph` used as a single node inside a bigger graph — from the outside, the bigger graph just sees one node that takes state in and returns an update, even though internally that "node" is quietly running its own multi-step graph. The reason to do this is the same reason Doc01 pulled config and logging into their own files instead of leaving everything in one script: a smaller, self-contained piece is easier to test alone, easier to reuse in more than one place, and easier to reason about without holding the whole system in your head at once.

How it really works

- Compile the smaller graph exactly as normal: `smaller_graph = builder.compile()`. Add it to the bigger graph the same way you'd add any function node: `bigger_builder.add_node("build_task", smaller_graph)` — a compiled graph satisfies the same "state in, update out" contract as a plain function.
- This only works when the smaller graph's state shape and the bigger graph's state shape **share** the fields that need to pass between them — the subgraph reads and writes whatever overlapping keys exist, exactly like any other node.
- A subgraph gets its own checkpoint entries nested under the parent's — pausing the parent graph mid-subgraph, and resuming it later, works the same way pausing anywhere else does, because supersteps and checkpoints don't care whether a node is a plain function or a whole graph.
- This document's own Build Task skeleton is a natural subgraph candidate — instead of copy-pasting its nodes into a bigger graph later, you can wrap the whole tested skeleton as one subgraph node and plug it straight in.
- Reach for a subgraph once a self-contained piece has enough of its own internal branching that inlining it would make the parent graph's diagram noisy — not for every two-node grouping.

Situation	What to do	Why
A self-contained piece with its own branching	Compile it separately, add it as a subgraph node	Keeps the parent graph's diagram readable
A tested, working skeleton (like this doc's Build Task)	Wrap it whole, plug it into the bigger graph	Already correct — no need to copy-paste and re-verify
Two or three nodes with no internal branching	Inline them in the parent graph	A subgraph adds ceremony a plain node pair doesn't need
Parent and child state share no fields	Not a subgraph — pass data another way	A subgraph node only exchanges data through shared state keys

Common mistakes:

- *Mistake:* wrapping every small group of nodes as a subgraph "for tidiness." → *Symptom:* more files and compiled objects to track, with no real gain in testability or reuse. → *Fix:* reach for a subgraph when a piece has its own real branching or gets reused in more than one place — not as a default habit.
- *Mistake:* giving the subgraph a state shape that shares no fields with the parent. → *Symptom:* the subgraph runs, but nothing it did shows up in the parent's final state. → *Fix:* design the subgraph's state to include the exact keys the parent needs back.

Where you'll meet it: this document's own Build Task is designed to be reusable this way once Doc10 grows it into Project 3. Doc11 sometimes wraps one specialist's whole internal reasoning as a subgraph node under a supervisor, keeping the top-level diagram to "who talks to whom" instead of every specialist's internals.

Quick cheat sheet:

- `compile()` a smaller graph, then `add_node(name, that_compiled_graph)` — same contract as any node.
- Parent and child state must share the fields that need to cross between them.
- Checkpoints and pausing work the same inside a subgraph as anywhere else.
- Use it for a genuinely self-contained, reusable, or internally-branching piece — not every small group of nodes.

Streaming a graph's execution

`.invoke()` runs the whole graph and hands you a result only once everything is finished — fine for a script, a poor fit for anything a person is watching happen live. `.stream()` instead gives you each node's output as it happens, one step at a time, while the graph is still running — the difference between a chat UI that shows "searching your documents… thinking… writing an answer…" and one that just shows a blank screen for however long the whole graph takes.

How it really works

- `for step in graph.stream(input):` yields one item per finished node — each item is the update that node just produced, in the same shape a plain `.invoke()` would have merged into state.
- Streaming modes differ in what you get per step: `"updates"` gives just the new keys a node returned; `"values"` gives the whole state as of that step; `"messages"` (when nodes emit chat messages) streams individual LLM tokens as they're generated, not just whole node outputs — the mode a real chat UI actually wants, the graph-level version of Doc04's streaming topic.
- Async graphs stream with `astream()` the same way an async client streams tokens — this is Doc08b's event loop doing the same job it does for any other long-running I/O, just wrapping a whole graph run instead of one API call.
- Streaming and checkpointing are independent — a graph can stream its progress and still not be checkpointed, or the reverse. Don't assume seeing live updates means the state is being saved anywhere durable.
- A production UI usually wants both: `"messages"` mode for live token-by-token text, and a final read of the completed state for anything structured the UI needs after the run finishes.

Mode	What you get per step	Use when
`"updates"`	Just the keys the node just returned	Logging, or a progress indicator ("step 3 of 5 done")
`"values"`	The whole state as of that step	Debugging — see everything at every point
`"messages"`	Individual LLM tokens as they generate	A chat UI showing the answer typing out live

Mode	What you get per step	Use when
`.invoke()` (no streaming)	Nothing until the whole run finishes	A script, a batch job, anything nobody is watching live

Common mistakes:

- *Mistake:* assuming streaming implies the state is being saved somewhere durable. → *Symptom:* a "live" run that still loses everything on a restart. → *Fix:* streaming and checkpointing are separate concerns — attach a real checkpointer if the run must survive a restart.
- *Mistake:* using `"updates"` mode to try to show token-by-token text. → *Symptom:* the UI updates in big chunks per node, not smoothly per word. → *Fix:* `"messages"` mode is the one built for token-level streaming.

Where you'll meet it: Doc04's streaming topic is the same idea for a single model call, one layer down. Doc08b's event loop is the mechanism `astream()` runs on. Project 3 and Project 5 both show a user which agent is working via streamed progress instead of one long silent wait.

Quick cheat sheet:

- `.invoke()` waits for everything; `.stream()` yields as each node finishes.
- `"updates"` = just new keys; `"values"` = whole state; `"messages"` = live tokens.
- `astream()` for async graphs — same event loop as any other async I/O.
- Streaming ≠ checkpointing — a live run can still not be saved anywhere.

Debugging a graph visually

LangGraph can draw an actual picture of the graph you built — its nodes, its edges, and which edges are conditional — instead of you reading a pile of `add_node` / `add_edge` calls and reconstructing the shape in your head. A graph with a wrong or missing edge is almost always faster to spot by looking at the diagram than by reading code line by line: a node with no arrow leaving it, or an edge pointing to the wrong place, jumps out visually in a way it never does in text.

How it really works

- `graph.get_graph().draw_mermaid()` returns Mermaid text — paste it into a Mermaid renderer, or most Markdown viewers that support Mermaid — showing every node and edge in your **compiled** graph. `draw_mermaid_png()` gives an image file directly, useful for a bug report or a design doc.
- The diagram reflects what you actually built, not what you intended — a conditional edge missing one of its mapped destinations, or a node with no outgoing edge at all, shows up as a dead end in the picture exactly as it would in a real stuck run.
- Draw the diagram **before** running a graph that isn't behaving as expected, not after — "is this graph actually shaped the way I think it is?" is usually answerable in seconds from the picture, versus minutes of re-reading `add_edge` calls.

- Combine the diagram with `.stream()`'s step-by-step output to answer two different questions: the diagram shows what **could** happen; the stream shows what **did** happen on one specific run.
- This is the graph-shaped version of the general debugging discipline Doc14 teaches: find the smallest artifact that shows you the real shape of the problem, before reading code line by line.

Question	Tool	Why
"Is my graph shaped the way I think?"	`draw_mermaid()` before running anything	A wrong edge is visible in seconds
"What actually happened on this one run?"	`.stream()`'s step-by-step output	The diagram shows what could happen, not what did
"Where exactly did it die?"	The last checkpoint before the failure	State at that point tells you which node's assumptions broke
Sharing the shape with a teammate or in a bug report	`draw_mermaid_png()`	A picture, not a wall of `add_edge` calls

Common mistakes:

- *Mistake:* debugging a wrong-looking run by re-reading `add_node` / `add_edge` calls line by line first. → *Symptom:* minutes spent mentally reconstructing a shape one Mermaid call would show in seconds. → *Fix:* draw the diagram first, every time a graph isn't behaving as expected.
- *Mistake:* drawing the diagram once, early, and never again as the graph grows. → *Symptom:* the mental picture of the graph quietly drifts from the real one after a few edits. → *Fix:* re-draw it whenever you add a node or an edge you're not fully sure about.

Where you'll meet it: Doc14 uses this as its primary tool for graphs whose edges are wrong. Project 3's "Plan Before You Code" step asks you to sketch the graph on paper first — `draw_mermaid()` is how you check that sketch against the real, compiled thing afterward.

Quick cheat sheet:

- `graph.get_graph().draw_mermaid()` — a picture of your compiled graph, nodes and edges.
- Draw it **before** debugging a strange run, not after re-reading all the code.
- The diagram shows what could happen; `.stream()` shows what did happen on one run.
- Re-draw it as the graph grows — a stale mental picture is a common source of "wait, why did it go there?"

Long-term memory: remembering across separate conversations

A checkpointer persists one conversation's state so that same conversation can pause and resume — but a brand-new conversation, a new `thread_id`, still starts with a brand-new, empty state, checkpointer or not. Long-term memory is a different, separate thing: a store that survives across different conversations entirely, so the same user, talking to the agent again next week in a totally

new thread, finds the agent still remembers their name or a preference they stated last time. Doc07 named this gap early and deliberately left it unsolved, because at that point there was no graph to hang the real mechanism on. Now there is.

How it really works

- A checkpointer is keyed by `thread_id` **on purpose**, so one user's in-progress conversation never leaks into another's. Long-term memory has to live outside that boundary, keyed by something that outlives any one thread — usually a user id.
- `langgraph.store`'s `Store` interface (`InMemoryStore` for local dev, `PostgresStore` for production) holds items under a **namespace**, typically `(user_id, "memories")`. A node calls `store.put(namespace, key, value)` to save a fact, `store.get(namespace, key)` to fetch one, or `store.search(namespace, query=...)` to find relevant ones — all independent of whatever thread the current run belongs to.
- The usual pattern: a node reads from the store at the **start** of a run (load what's already known about this user) and writes to it during or after the run (save anything new worth keeping) — two separate touches to the same store, in the same run.
- This directly contrasts with Doc03's "no memory inside the model": the model itself never remembers anything between calls, whether it's this run's scratchpad, a checkpointed thread, or a long-term fact — every one of those is context your code re-injects. `langgraph.store` is one more place that re-injection can come from, alongside the checkpointer and the scratchpad.
- **What's worth saving is the real design question**, not the mechanism. You don't want a full transcript of every past conversation dumped back into every future run — neither useful nor readable. You want genuinely durable facts: a name, a stated preference, a past decision. Deciding what's worth remembering is itself often a small, cheap model call: "does this message contain a fact worth remembering long-term?" — write to the store only when the answer is yes.
- A long-term store is still just a database from a safety standpoint — the same metadata-filtering discipline Doc08 teaches for tenant scoping applies to a namespace keyed by user id: never let a model-chosen argument decide whose namespace gets read or written.

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()
namespace = (user_id, "memories")
store.put(namespace, "name_preference", {"text": "Prefers to be called Sam"})

remembered = store.search(namespace, query="how should I address this user?")
```

	Checkpointer (this document, above)	Long-term store (`langgraph.store`)
Keyed by	`thread_id` — one conversation	A durable id — usually `user_id`

	Checkpointer (this document, above)	Long-term store (`langgraph.store`)
Lives for	That conversation, pause-to-resume	Across every future conversation
Holds	The whole run's state	Curated facts someone decided were worth keeping
Written	Automatically, every superstep	Deliberately, by a node — often gated by its own small model call

Common mistakes:

- *Mistake:* assuming a longer-lived checkpointer is the same thing as long-term memory. → *Symptom:* a new conversation with the same user still starts from zero, because it's a new `thread_id`. → *Fix:* long-term facts need a separate store keyed by user id, read and written on purpose — a checkpointer alone will never bridge across threads.
- *Mistake:* saving every message "just in case" instead of deciding what's worth keeping. → *Symptom:* an unreadable dump nobody can use, and future runs either ignore it or drown in it. → *Fix:* gate writes behind a real decision — often a small, cheap model call asking whether this specific fact is worth remembering.

Where you'll meet it: Doc07's agent-memory topic names this exact gap before a graph exists to fill it. Doc03's no-memory fact is what long-term memory, like every other kind, works around. Project 10 — MemoryKeeper builds this exact system end to end, checkpointer and long-term store side by side.

Quick cheat sheet:

- A checkpointer is per-thread; long-term memory is per-user, across threads.
- `langgraph.store`: `put` / `get` / `search` under a `(user_id, "memories")`-style namespace.
- Read at the start of a run, write during or after — two separate touches, same store.
- Decide what's worth saving on purpose — often with a small model call — never "everything, just in case."
- Never let a model-chosen value pick whose namespace gets read or written.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- LangGraph documentation (home) — start with the Quickstart/tutorials.
- LangGraph GitHub repo — the `examples/` folder has real graphs to read.
- LangChain Academy — has a free LangGraph course; do it alongside this document.

Practice Exercises

Setup: same venv as before — if it's not active, `cd 09_langgraph && source ../01_python_foundations/.venv/bin/activate` (or your own venv for this folder). New packages for this document: `pip install langgraph langchain-openai langgraph-checkpoint-sqlite` (the last one for `SqliteSaver`).

Where your code lives: all of it under `09_langgraph/practice/` (`mkdir -p practice`), never loose beside this README. Exercises are grouped **by topic, not by level** — the same convention as Doc01/02/07/08 — so one topic's growth from basic to intermediate stays visible in one file.

The full file layout, all exercises:

```
practice/
├── agent.py ... tools.py              Doc07's Build Task files, copied in
│                                       (for the Real-world exercise)
├── agent_loop_to_graph_practice.py    Real-world
├── approval_graph.py                  Failure (Approach 3)
├── conditional_routing_practice.py    Intermediate + Edge cases
│                                       (two sections)
├── first_graph_practice.py            Basic
├── loop_limit_interrupt_practice.py   Failure
├── pause_script.py                    Failure (Approaches 2-3)
└── resume_script.py                   Failure (Approaches 2-3)
```

Why each script exists:

- `agent_loop_to_graph_practice.py` — proves a graph can do exactly what Doc07's hand-built loop did — the direct rehearsal for the Build Task.
- `approval_graph.py`, `pause_script.py`, `resume_script.py` — the pause/resume across two real processes: one module for the graph, one script that pauses, one that resumes.
- `conditional_routing_practice.py` — real branching, plus what happens when a routing function returns a value nothing handles.
- `first_graph_practice.py` — the 3 real ingredients (state, nodes, edges) at their smallest — every later exercise and the Build Task assume you already have this working.
- `loop_limit_interrupt_practice.py` — the only place you watch an unguarded loop actually hit its limit, and a paused graph resume in the same process.

For this document, save your practice code as:

- **Basic** (your first graph) is its own topic — save it as `practice/first_graph_practice.py`.
- **Intermediate** (real branching) and **Edge cases** (an unhandled routing value) are both about conditional edges — save them together as `practice/conditional_routing_practice.py`, one section per level.

- **Real-world** (rebuild Project 2 as a graph) is its own topic — save it as `practice/agent_loop_to_graph_practice.py`.
- **Failure** (an endless loop, and a real pause/resume) is its own topic — save it as `practice/loop_limit_interrupt_practice.py`.

Jump to an exercise: Basic · Intermediate · Real-world · Edge cases · Failure · Build Task

Basic — your first graph

- **What:** a 2-node `StateGraph` with no branching, tracing by hand what the state looks like after each node.
- **Why:** you need the simplest possible graph working, with a mental model you've checked by hand, before adding anything that could hide a misunderstanding.
- **How to code it:** define a `TypedDict` state with one field, two node functions that each update it, `add_node(...)` for both, `add_edge(...)` connecting them, then `compile().invoke({...})` and print the result.
- **Save as:** `practice/first_graph_practice.py`.
- **Used later by:** the Build Task's `state.py` / `nodes.py` / `graph.py`, and Project 3's own Step 1, which is this exact exercise for real.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — real branching

- **What:** a conditional edge that picks a path based on a field in state, tested on both paths.
- **Why:** this is the mechanism every "should I search / should I call this agent" decision in the rest of the curriculum is built from.
- **How to code it:** write a `route(state)` function that returns `"path_a"` when `state["flag"]` is true and `"path_b"` otherwise (a plain `if` / `else`), wire it with `add_conditional_edges("node", route, {"path_a": "...", "path_b": "..."})`, and invoke with both flag values.
- **Save as:** `practice/conditional_routing_practice.py`, under an `# Intermediate` section (this file also holds the Edge cases exercise below, in its own section).
- **Builds on:** the Basic graph — same shape, now with a real decision.
- **Used later by:** Project 3's "search or don't" routing, and Project 4's supervisor routing.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — rebuild Project 2 as a graph

- **What:** rebuild Project 2's agent (Doc07) as a graph with the same tools — same behavior, a new, clearer build.
- **Why:** this is the exact translation exercise (loop → graph) this document's Build Task needs — do a first pass here, on a system you already understand completely.

- **How to code it:** copy Doc07's Build Task files into `practice/`. Turn the loop into a `think` node (the model call with `bind_tools`) and an `act` node (Doc07's `run_tool()` for every tool call), with a conditional edge deciding "act" vs. "finish". Add a `recursion_limit`, then confirm the graph uses the same tools as Doc07's `run_agent()` on the same test prompts, and count model turns.
- **Save as:** `practice/agent_loop_to_graph_practice.py`.
- **Builds on:** Doc07's hand-built ReAct loop and its `create_agent` comparison.
- **Used later by:** the Build Task, and Project 3's Step 2, which does this exact migration for real, with a checkpointer added.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Edge cases — an unhandled routing value

- **What:** a conditional edge whose check function returns a value that doesn't match any known path — see what happens.
- **Why:** you want to know, before it happens in a real run, whether this fails loudly (good) or silently does something unexpected (bad) — and if it's the latter, you need to guard against it yourself.
- **How to code it:** deliberately have your routing function sometimes return a string not in your `add_conditional_edges` mapping, run it, and read exactly what LangGraph does.
- **Save as:** `practice/conditional_routing_practice.py`, under an `# Edge cases` section (this file also holds the Intermediate exercise above, in its own section).
- **Builds on:** the Intermediate section of this same file.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Failure — an endless loop, and a real pause/resume

- **What:** a graph with a loop that has no exit rule, run in a controlled, limited way. Then a checkpointer connected, proving state survives an `interrupt()` → resume cycle.
- **Why:** both of these are things you need to have actually watched happen once — an endless graph loop, and a real, working pause that resumes correctly — before you trust yourself to build them safely in a real project.
- **How to code it:** build a 2-node cycle with no conditional exit, run it with a low `recursion_limit` so it fails fast instead of hanging. Then, separately, add `checkpointer=MemorySaver()`, call `interrupt()` in one node, invoke once (it pauses), then resume with the same `thread_id` in a **fresh Python process** — with `MemorySaver` it fails, with `SqliteSaver` it works — using `graph.get_state(config)` to see what's paused first.
- **Save as:** `practice/loop_limit_interrupt_practice.py`.
- **Builds on:** the layered-limits idea from Loops, and the Checkpointers/`interrupt()` topics.
- **Used later by:** the Build Task's `interrupt()` requirement, and Project 3's Approval agent, which is this exact gate for real.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Build Task — Graph Skeleton (feeds into Project 3)

Stuck on the Build Task? Hint 1 · Hint 2 · Show me the solution

Goal: the graph foundation Doc10 builds into the full Project 3 app. This is not its own project — it's the skeleton Project 3 — DocuMind copies straight into its own `state.py` / `nodes.py` / `graph.py` and grows from there.

Requirements:

- A `StateGraph` with a typed state shape.
- At least one conditional edge (not just a straight line of nodes).
- A checkpointer connected — state has to survive a pause, tested from a fresh process.
- At least one `interrupt()` point, standing in for human checking before a "risky" action (the action itself can just be a placeholder for now).
- Rebuilds Project 2's tool-calling behavior as graph nodes (proves you understand turning a loop into a graph).

Inputs: the same free-text task input as Project 2. **Outputs:** the same final-answer behavior as Project 2, plus a visible log of the state, and a working pause/resume cycle. **Constraints:** no node may quietly swallow an error — failures must show up in state or logs.

```
09_langgraph/practice/build_task/
├── agent.py             Doc07's: AGENT_TOOLS, run_tool(), run_agent()
├── docs_search.py       Doc07's, unchanged
├── exceptions.py        Doc07's (+ UnhandledRouteError)
├── graph.py             build_graph(checkpointer=None) -> compiled graph
├── http_client.py       Doc07's, unchanged
├── logging_setup.py     Doc07's, unchanged
├── nodes.py             think, act, risky_action, route
├── pause_demo.py        pauses a refund and saves it to SQLite
├── refunds.py           issue_refund -- the placeholder risky tool
├── resume_demo.py       a new process: finds the pause and resumes it
├── state.py             AgentState -- the typed state shape
├── test_graph.py        proves the Test Cases below
└── tools.py             Doc07's, unchanged
```

- `agent.py`, `docs_search.py`, `http_client.py`, `logging_setup.py`, `tools.py` — **What/Why:** Doc07's Build Task files, copied in unchanged — the graph reuses Doc07's tools and `run_tool()` instead of writing them again.
- `exceptions.py` — **What/Why:** Doc07's file plus `UnhandledRouteError`, so an unexpected route value fails with a clear, named error.
- `graph.py` — **What/Why:** `build_graph(checkpointer=None)` wires the nodes, the 3-way edge and the checkpointer — the caller chooses where checkpoints live.

- `nodes.py` — **What/Why:** `think` / `act` / `risky_action` / `route` — Doc07's loop rebuilt as graph functions, plus the one `interrupt()` point before a risky tool runs.
- `pause_demo.py` — **What/Why:** starts a refund task, stops at the approval question, saves to `checkpoints.sqlite`, and exits.
- `refunds.py` — **What/Why:** `issue_refund(order_id)`, a placeholder `@tool` standing in for an action with real consequences — the thing worth pausing for.
- `resume_demo.py` — **What/Why:** a brand-new process that finds the paused run with `get_state()` and resumes it — the "from a fresh process" Test Case.
- `state.py` — **What/Why:** one typed shape listing everything any node can read or write — the graph's whole public surface, in one place.
- `test_graph.py` — **What/Why:** proves the Test Cases below, each printing `OK` or `WRONG` — including a rejected refund and a bad route value.
- `tools.py` — **What/Why:** Doc07's tool library, imported unchanged.

Run it: `cd practice/build_task && python test_graph.py` — from inside the folder, so `from graph import build_graph` finds the file next to it. Then `python pause_demo.py`, and after it exits, `python resume_demo.py`.

Builds on: the Real-world loop-to-graph rebuild (state, `think`, `act`, `route`), the Intermediate conditional edge, the Edge cases `UnhandledRouteError`, and the Failure exercise's `interrupt()`, `SqliteSaver` and `get_state()` — **copy** them into this folder rather than starting from scratch. The tools and `run_tool()` come from Doc07's Build Task, unchanged.

Used later by: Project 3 — DocuMind's Step 1 and Step 2 are this skeleton's patterns, rebuilt directly into the project's own files — Step 1 is this document's Basic exercise (state, nodes, edges), Step 2 is its checkpointer + `interrupt()` cycle with `build_graph(checkpointer)` and `SqliteSaver`, later grown with a Retriever, a Reasoner, and an Approval agent.

Expected Behavior

- You can draw the node/edge diagram from the code (or the other way around), without running it.
- The graph reaches the same final answers as Project 2, for the same test tasks.
- Pausing at the `interrupt()` point and resuming later (from a fresh process, not just the same one still running) gives the correct, continued behavior, using the saved state.

Test Cases

Scenario	Expected
Task with no risky action	The graph finishes without ever hitting the interrupt

Scenario	Expected
Task that reaches the interrupt point	The run pauses, state is saved
Resume after interrupt (approved), from a fresh process	Graph continues from the saved state, correct final answer
Resume after interrupt (rejected)	The risky tool never runs; the answer says it wasn't approved
Conditional edge with an unexpected state value	Fails clearly and loudly, not a silent wrong path

Break-It / Debug Preview

- An endless loop from a missing or wrong conditional edge.
- State that doesn't actually survive an interrupt (a badly set up checkpointer).
- A node that changes state in a way that breaks a later node's assumptions.
- Full debugging drill in 14_debugging_lab.

Interview Topics Preview

- Why clear state beats hidden agent memory · checkpointer trade-offs (in-memory vs. saved) · when a graph is overkill vs. actually needed vs. a plain `create_agent` loop is enough.

Move On When

You can draw a graph's diagram from its code (or the other way), without running it, and the pause/resume cycle works. Full details: CURRICULUM.md §4.

Stuck? Ask for **Hint 1** through **Hint 2**. Ask for the full solution only if you say **"Show me the solution."**